



İZMİR UNIVERSITY OF ECONOMICS

CE316

Integrated Assignment Environment Design Document

Authors (Team #7):

Salih MALEKER

Kuzey ALDEMİR

Berke İKİKARAKAYALI

Ahmet Fatih ERGENEKON

Arda BARUT

Kutay KARAHAN

Date:

May 2, 2026

Contents

1	Overview	2
2	System and Database	2
2.1	Schema	2
2.2	Entity-Relationship Diagram	3
2.3	Use Case Diagram	4
3	Configuration Manager	4
3.1	How Configurations Work	4
3.2	Classes	5
3.3	Class Diagram	6
3.4	Import and Export	6
3.5	State Diagram	7
4	File and Directory Manager	7
4.1	Responsibilities	8
4.2	Classes	8
4.3	Processing Algorithm (How It Works)	8
4.4	Error Handling	9
4.5	Interaction with Other Modules	9
4.6	Activity Diagram	10
5	Execution Engine	10
5.1	Run Arguments and Command Construction	11
5.2	Timeout Configuration	11
5.3	Classes	11
5.4	How Execution Works	12
5.5	Class Diagram	13
5.6	Error Handling	13
5.7	Interaction with Other Modules	13
5.8	Sequence Diagram	14
6	Evaluation and Reporting	14
6.1	How Comparison Works	14
6.2	Classes	15
6.3	Class Diagram	16
7	UI and Deployment	16
7.1	Installer and System Deployment	16
7.2	Help	17
7.3	Screens	18
7.4	Controller	18
7.5	UI Mockups	19
7.6	Possible Problems and Risks	20

1 Overview

The Integrated Assignment Environment (IAE) is a Windows desktop application for managing and evaluating programming assignments. The lecturer creates a project, selects a language configuration, and points the application to a folder of student ZIP files. When the lecturer clicks Run, the IAE extracts every ZIP, compiles or runs the code inside, compares the output to the expected answer, and saves a result for each student.

The application does not use any external server. Each project is saved as a single SQLite file that the lecturer can open, close, or copy. A Windows installer packages the application with all required libraries, so the lecturer does not need to install anything extra.

2 System and Database

This component manages how projects are created, opened, and saved. It covers Requirement 3 and Requirement 10.

Every project is stored in a single SQLite file with the `.iae` extension. SQLite was chosen because it is file-based and does not need a server. The lecturer can move or copy the file like any other document. When a project is opened, the application reads the file and restores everything: the configuration, the student submissions, and the evaluation results from previous runs. When the project is saved, everything is written back to the same file.

Requirement 3 (user requirement): The user must be able to create a project that uses an existing or a new configuration.

Requirement 10 (user requirement): The user must be able to open and save projects to operate on them at any time.

2.1 Schema

The database has four tables.

The **CONFIGURATION** table stores language rules that can be reused across multiple projects. Each row holds the language name, the file extension for student source files, the path to the compiler or interpreter, compile-step arguments, and run-step arguments. The expected output path and comparison mode are not stored here because they change per assignment, not per language.

The **PROJECT** table represents one assignment. It links to a configuration and also stores the expected output file path, the run arguments for the test case, the timeout values for compilation and execution, and the normalization mode chosen for this assignment. One `.iae` file has exactly one **PROJECT** row.

The **STUDENT_SUBMISSION** table tracks each student's files after extraction. It stores the original ZIP path, the folder created during extraction, and the path to the source file

found inside. This table is filled by the File and Directory Manager when the lecturer runs the project.

The `EVALUATION_RESULT` table stores the outcome for each student. It holds the comparison status, which is one of `PASS`, `FAIL`, `ERROR`, `COMPILE_ERROR`, or `TIMEOUT`. It also stores the actual output from the student's program, the expected output, the list of lines that differed as a JSON array, any error messages, the normalization mode that was used, and a timestamp.

2.2 Entity-Relationship Diagram

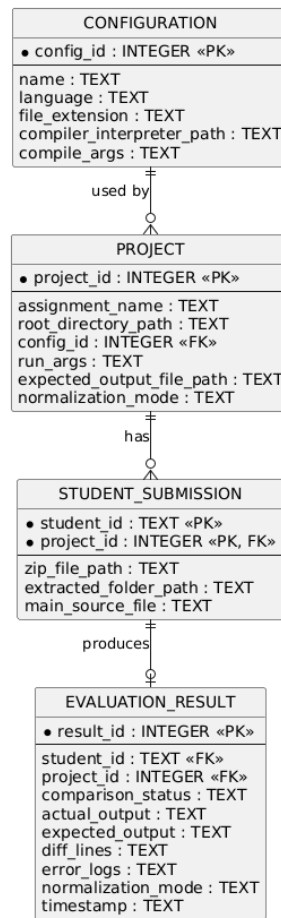


Figure 1: Entity-Relationship Diagram

2.3 Use Case Diagram

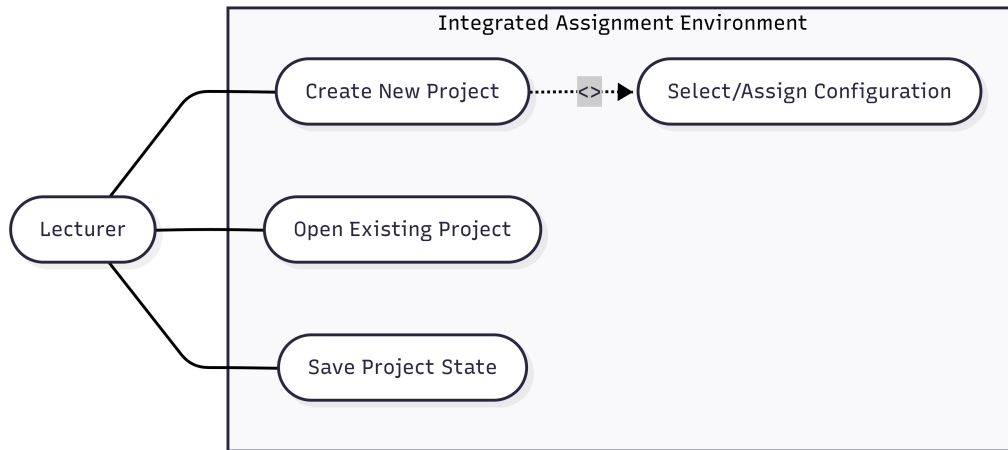


Figure 2: Use Case Diagram

3 Configuration Manager

This component is responsible for how the IAE handles different programming languages. It covers Requirement 4 and Requirement 5.

It stores the compiler or interpreter path, the file extension for student files, and the command-line arguments for the compile and run steps. When Execution Engine processes a Java file, it asks and gets the `javac` and `java` commands. The Execution Engine has no language-specific knowledge of its own.

`ConfigurationService` stores, retrieves, and validates language configurations, and `ImportExportService` lets the lecturer export configurations to a file and import them on another machine.

Requirement 4 (user requirement): The user must be able to create, edit and remove a configuration.

Requirement 5 (user requirement): The user must be able to import and export configurations.

3.1 How Configurations Work

Each language is described by a `LanguageConfig` object. It holds the language name, file extension, compiler path, and an argument list for the compile step. This separation makes the design work for all language types. Python has an empty compile list since it is interpreted. Java uses `javac` to compile and `java` to run. C uses `gcc` to compile and runs the binary directly.

`ConfigurationService` keeps all configs in a `Map` where the key is the language name. It has methods like `getConfigForLanguage(name)`, `addConfig(config)`, `updateConfig(config)`, `removeConfig(name)`, and `listAll()`. When the Execution Engine needs to process a submission, it calls `getConfigForLanguage()` to get the right config. `ConfigurationService` also has `loadFromProject()` and `saveToProject()` methods so configs are stored in the database and reloaded when the project is opened.

Before a config is used, `validate()` checks that the compiler path exists and the argument lists are correct. If the check fails, the UI shows a warning so the lecturer can fix the path before running anything.

3.2 Classes

`ConfigStatus` is an enum with three values: `VALID`, `INVALID_PATH`, and `INVALID_ARGS`. Every config has one of these values after `validate()` is called.

`LanguageConfig` is the main data class. It holds `name`, `fileExtension`, `compilerPath`, `compileArgs` as a `List<String>`, `runArgs` as a `List<String>`, and the current `ConfigStatus`. It has a `validate()` method that checks the path and arguments, updates the status field, and returns a `ValidationResult`. It also has `buildCompileCommand()` and `buildRunCommand()` methods that the Execution Engine uses to get the final command for a student file.

`ValidationResult` is the object that `validate()` returns. It has a `ConfigStatus` and a message string. The message is empty when the status is `VALID`, and it contains an explanation like "Compiler not found at /usr/bin/gcc" when something is wrong. It also has an `isValid()` method. Using a result object instead of throwing an exception is better here because a failed validation is a normal situation during import, not an error.

`ConfigurationService` manages all `LanguageConfig` objects for one project. Its public methods are `getConfigForLanguage(name: String): LanguageConfig`, `addConfig(config: LanguageConfig): void`, `updateConfig(config: LanguageConfig): void`, `removeConfig(name: String): void`, and `listAll(): List<LanguageConfig>`. Inside it uses a `Map<String, LanguageConfig>` with the language name as the key.

`ImportExportService` handles moving configurations between machines. It has two methods: `exportToFile(path: String): void` and `importFromFile(path: String): ImportResult`. Both use a JSON file that contains a list of serialized `LanguageConfig` objects.

`ImportResult` is what `importFromFile()` returns. It has a boolean success field, a list of configs that were successfully imported, a list of `SkippedEntry` objects for configs that failed, and an error message if the whole operation failed. The UI uses this to show the lecturer what happened after the import.

`SkippedEntry` is a small class that holds a `LanguageConfig` and a reason string. The reason comes from `ValidationResult.message` and explains why that config was not imported, for example "compiler path not found on this machine".

3.3 Class Diagram

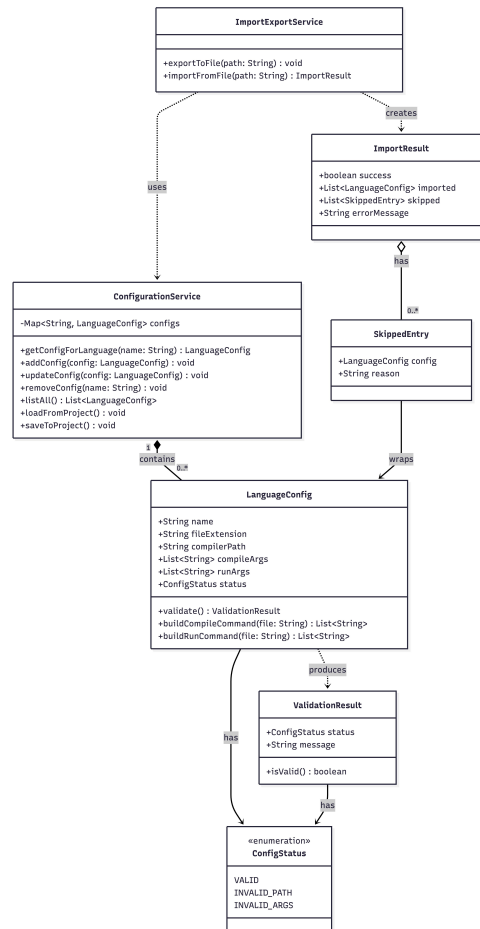


Figure 3: Configuration Manager Class Diagram

3.4 Import and Export

A lecturer may set up configurations on their office computer and want the same setup on a lab computer. Instead of creating everything again, they can export their configs to a file, copy it to the other machine, and import it.

Export is simple. `exportToFile()` takes all **LanguageConfig** objects from **ConfigurationService**, converts them to JSON, and writes the file to disk. The compiler paths are saved as they are, so they may not work on a different machine.

Import is more careful. `importFromFile()` reads the JSON file and calls `validate()` on each config before adding it. Configs that pass are added to **ConfigurationService**. Configs that fail, usually because the compiler path does not exist on the new machine, are put into a **SkippedEntry** and shown to the lecturer. The lecturer can fix the path manually after the import.

Import has two modes. In merge mode, which is the default, a config with the same language name replaces the existing one and new configs are added. In replace mode, all existing configs are deleted first. The lecturer picks the mode in the import dialog.

3.5 State Diagram

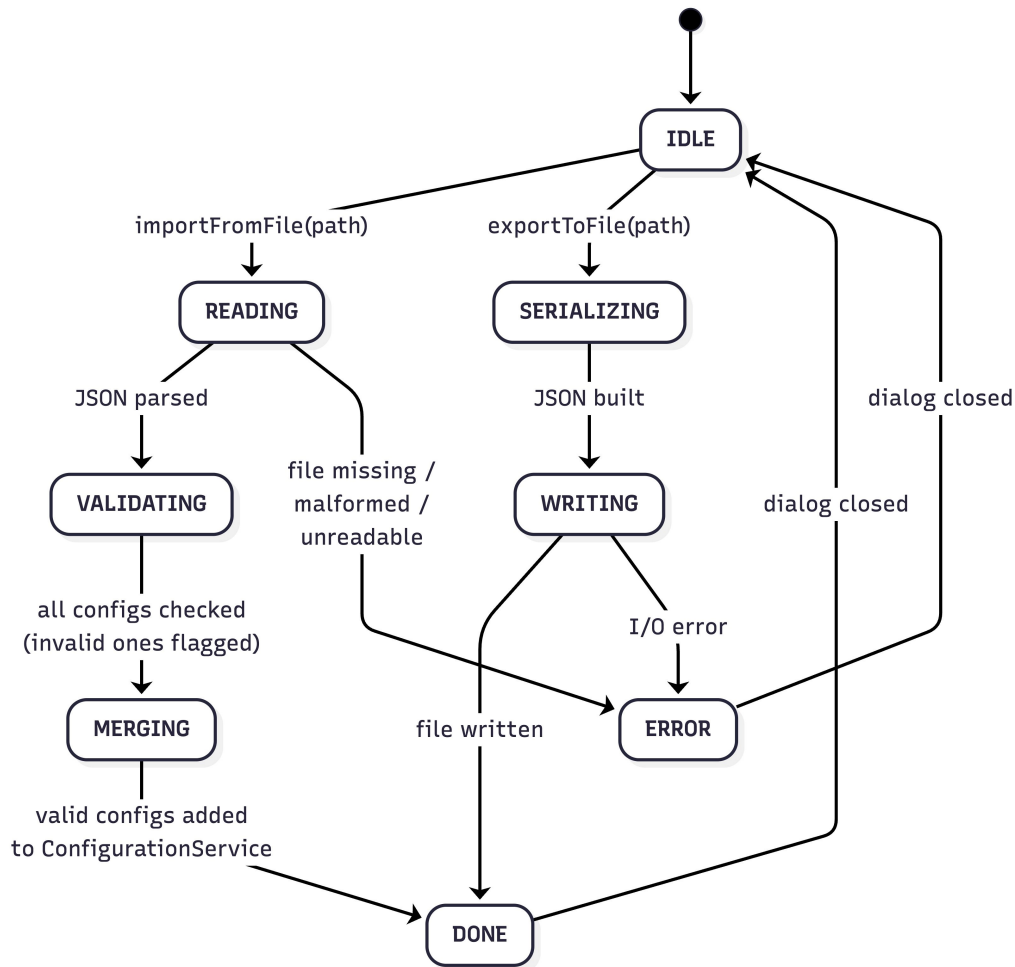


Figure 4: Import and Export State Diagram

4 File and Directory Manager

This component handles extracting student submissions and covers Requirement 6. The lecturer selects a folder that contains student ZIP files, and this component extracts all of them automatically. The extraction is never done one by one manually.

The project description clearly says the application cannot use `zip` or `unzip` from the command line. Because of this, extraction is done with Java's built-in `java.util.zip` library, and no external tool is needed.

Requirement 6 (user requirement): The software must be able to process ZIP files for each student.

4.1 Responsibilities

The File and Directory Manager performs the following tasks:

- Scanning the selected folder for uploaded ZIP files.
- Extracting ZIP contents into new folders using `java.util.zip`.
- Creating folder names based on student IDs.
- Finding the required main source file (like `main.c`) inside the extracted content.
- Handling broken files or errors without stopping the system.

4.2 Classes

We use two main classes to keep the design simple and clean:

1. **FileManager:** This class does the actual work. Its main method is `prepareSubmissions(zipDirectory: String, sourceFileName: String): List<Submission>`. This method scans the folder, processes each ZIP, and returns a list of `Submission` objects. It uses three private helper methods:

- `extractZip(zipPath: String, targetDir: String): boolean` — Extracts one ZIP file using `java.util.zip`. Returns false if the file is corrupt or unreadable.
- `resolveSourceFile(folder: String, fileName: String): String` — Searches the extracted folder recursively for the required source file. Returns null if not found.
- `extractStudentId(zipFile: String): String` — Removes the `.zip` extension from the filename to derive the student ID.

2. **Submission:** This is a data class that stores information about a student's files. The Execution Engine uses it later. It has four fixed fields:

- `studentId: String` — The ZIP filename without the extension.
- `zipFilePath: String` — The original location of the ZIP.
- `extractedFolderPath: String` — The new folder created after extraction.
- `mainSourceFile: String` — The full path to the source code file.

4.3 Processing Algorithm (How It Works)

Each ZIP file must be named after the student's ID (for example, `20210001.zip`). The algorithm processes each file one by one to ensure the system is stable.

Here is the pseudo-code for our logic:

```
for each zip_file in zipDirectory:
    student_id = extractStudentId(zip_file)
    target_folder = create_directory(student_id)

    extraction_success = extractZip(zip_file, target_folder)
```

```

if not extraction_success:
    log_error("Failed to extract", zip_file)
    continue // Move to the next student

main_file = resolveSourceFile(target_folder, sourceFileName)

if main_file is missing:
    log_error("Source file not found", zip_file)
    continue // Move to the next student

add new Submission(student_id, zip_file, target_folder, main_file) to list

```

4.4 Error Handling

To make sure the batch processing runs smoothly, the module handles these errors:

- Invalid or corrupted ZIP files.
- Missing source files inside the folder.

If any of these errors happen, the error is logged, and the processing continues directly with the next student's ZIP file without interrupting the whole run.

4.5 Interaction with Other Modules

This component acts as a bridge and works with the following parts of the system:

- **Controller:** Calls the `FileManager` when the lecturer clicks the “Run” button.
- **Execution Engine:** Receives the final list of `Submission` objects from here to compile and run them.

4.6 Activity Diagram

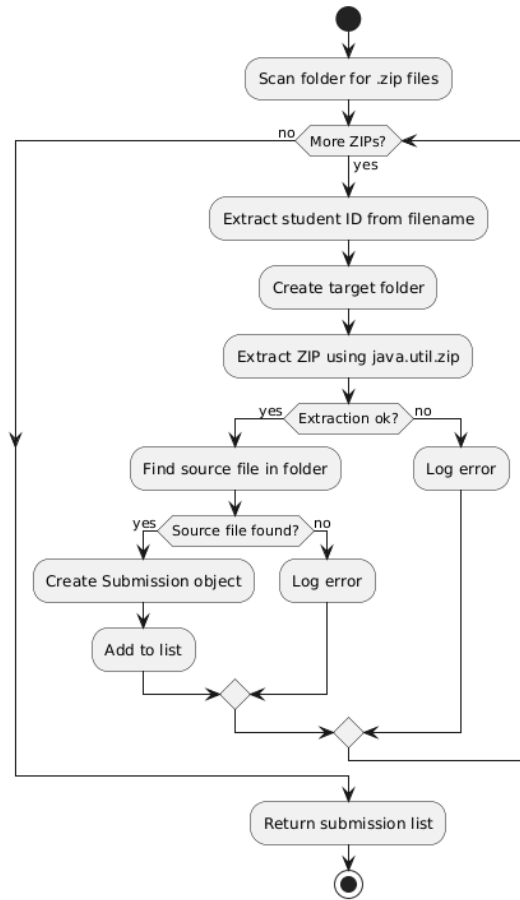


Figure 5: File Extraction Activity Diagram

5 Execution Engine

This component runs the student code and covers Requirement 7. It does not include a compiler. Instead it calls external tools like `gcc`, `javac`, or `python` using Java's `ProcessBuilder` and captures their output.

The language-specific commands all come from the `LanguageConfig` provided by the Configuration Manager, so this component does not have any hardcoded language rules. Each student is processed one by one and an error for one student does not stop the others.

Requirement 7 (user requirement): The software must be able to compile or interpret source code using the configuration of the project.

Execution Engine tasks include:

- Accepting execution requests from the Controller,
- Compiling source code using the correct compiler,

- Running the compiled or interpreted program,
- Returning execution results to the application,
- Ensuring the system continues execution in error conditions.

5.1 Run Arguments and Command Construction

The IAE must support passing command-line arguments to the student's executable, because the lecturer may provide a hidden input (for example, a list of strings to sort) that the student program reads from `argv`. These arguments are stored in the **PROJECT** table as a single `run_args` TEXT field and are entered by the lecturer in the Project Creation Wizard.

The final command is assembled in two stages. First, `LanguageConfig.buildRunCommand(workingDir, projectRunArgs)` builds the base command from the configuration's own run-step arguments (for example, `java Main` for a Java project) and then appends the tokenized `projectRunArgs` from the project. This keeps the language-specific invocation inside `LanguageConfig` while allowing the lecturer to set test-specific inputs at the project level.

5.2 Timeout Configuration

To prevent the system from hanging on infinite loops or blocking I/O, the Execution Engine enforces a time limit on every external process it spawns. Both timeout values are stored per project in the **PROJECT** table (`compile_timeout_sec` and `run_timeout_sec`) and are configurable by the lecturer in the Project Creation Wizard. The default values are 60 seconds for the compile step and 30 seconds for the run step. If a process exceeds its timeout, **Enforcer** kills it and returns an **Executed** result with `timedOut = true`, which the **Executioner** then records as a **TIMEOUT** status in the report.

5.3 Classes

The Execution Engine is made up of three classes.

Executioner coordinates the execution process. It loops through all project submissions and calls the correct compile and run operations. It holds a reference to **ConfigurationService** and **ReportingService**. Its public methods are:

- `executeAll(submissions: List<Submission>): void` — the entry point called by the Controller,
- `compile(submission: Submission): boolean` — attempts compilation and returns the success status,
- `run(submission: Submission): Executed` — executes the compiled program,
- `processResult(result: Executed): void` — forwards results to **ReportingService**.

Enforcer handles system-level process execution using Java's **ProcessBuilder**. It calls external tools, captures their output, and enforces the time limit. Its private fields are `command: String` and `workingDirectory: String`. Its public methods are:

- `execute(command: String): int` — runs the command and returns its exit code,
- `getOutput(): String` — returns the captured stdout,
- `getError(): String` — returns the captured stderr,
- `getExitCode(): int` — returns the last exit code.

`Executed` stores the result of one compile-and-run cycle. It has five fields:

- `compSuccess: boolean` — true if the compile step returned exit code 0 or was not needed.
- `runSuccess: boolean` — true if the run step returned exit code 0.
- `output: String` — the captured stdout.
- `errorMessage: String` — the captured stderr or compiler error text.
- `timedOut: boolean` — true if the process was killed by the timeout.

5.4 How Execution Works

When the lecturer clicks Run, the `Controller` calls `executeAll()` with the submission list. For each submission, the config is retrieved from `ConfigurationService`. If the configuration has compile arguments, the compile command is built with `buildCompileCommand()` and passed to `Enforcer` with the project's `compile_timeout_sec` value. If the exit code is not zero, a `COMPILE_ERROR` report is created and the loop moves to the next student.

If compilation is successful, or the language does not need a compile step, the run command is built with `buildRunCommand(workingDir, project.runArgs)` and executed with `run_timeout_sec`. If the process times out, a `TIMEOUT` report is created. Otherwise, the captured stdout is sent to `ComparisonService.compare()` with the expected output path and normalization mode from the project. The result is wrapped in a `StudentReport` and handed to `ReportingService.addReport()`.

Here is the pseudo-code for the execution logic:

```
for submission in submissions:                                # Executioner
    working_dir = get_directory(submission)                   # FileManager
    compile_status = run_command(
        config.compile_command, working_dir)                 # Enforcer
    if not compile_status:
        store_result(submission,
            "compile_error")                                  # Executed
        continue
    output = run_command(
        config.run_command,
        working_dir, config.arguments)                        # Enforcer
    result = evaluate(output)                                  # Evaluation
    store_result(submission, result)                           # Executed
```

5.5 Class Diagram

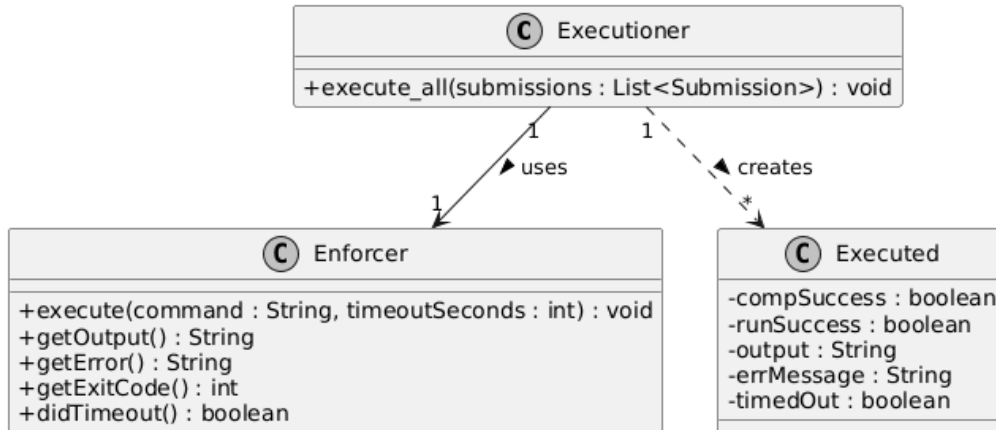


Figure 6: Execution Engine Class Diagram

5.6 Error Handling

Possible errors are:

- Missing or invalid files,
- Runtime issues,
- Compilation failures.

All errors are recorded, and processing continues with the next submission without interruption.

5.7 Interaction with Other Modules

The Execution Engine works together with other components:

- **Configuration Manager:** provides language-specific execution commands via `LanguageConfig`,
- **File and Directory Manager:** provides the list of `Submission` objects and working directories,
- **Evaluation and Reporting:** receives the program output for comparison and stores the final results,
- **Controller:** initiates the execution by calling `executeAll()`.

5.8 Sequence Diagram

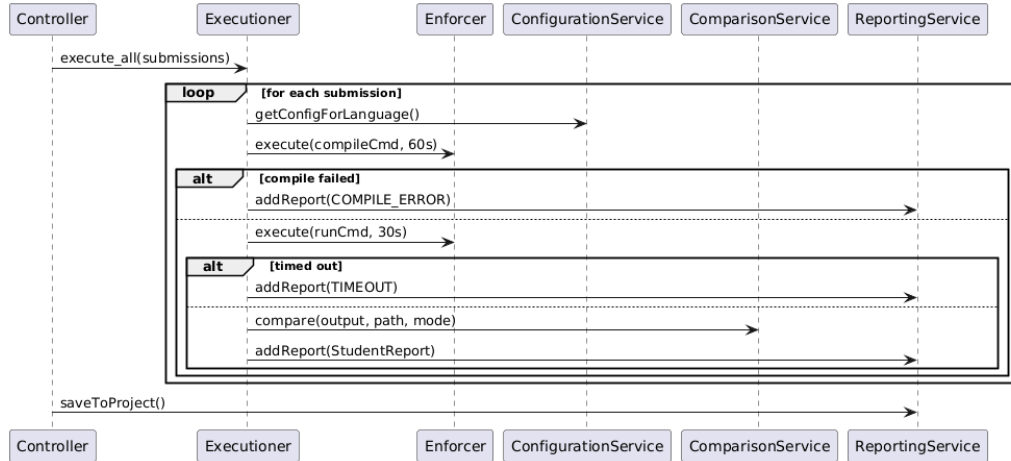


Figure 7: Execution Sequence Diagram

Execution starts when the user triggers the run action. The controller forwards this request to the Execution Engine, which loads the configuration and processes each submission sequentially. For every submission, the Execution Engine compiles the code using external tools, runs the resulting program with the project’s run arguments, and sends the output to the Evaluation and Reporting component.

6 Evaluation and Reporting

This component checks the student output against the expected answer and saves the results. It covers Requirement 8 and Requirement 9.

The component has two services. `ComparisonService` does the string comparison. `ReportingService` collects one `StudentReport` per student, saves everything to the database after the run, and gives the UI the summary and export methods it needs.

Requirement 8 (user requirement): The software must be able to compare the output of the student program and the expected output.

Requirement 9 (user requirement): The software must be able to display the results of each student file.

6.1 How Comparison Works

`ComparisonService.compare()` gets three inputs: the student program’s stdout, the path to the expected output file, and the normalization mode from the project. It reads the expected file from disk, normalizes both strings based on the mode, and checks if they are

equal. If they match, the result is `PASS`. If they do not match, the result is `FAIL` and the different lines are saved so the lecturer can see exactly what went wrong.

Normalization always converts Windows line endings to Unix line endings first. In `STRICT` mode that is all it does, so every character must match. In `TRIM_WHITESPACE` mode it also removes leading and trailing spaces from each line. In `CASE_INSENSITIVE` mode it applies the whitespace trimming and then converts both strings to lowercase before comparing.

`COMPILE_ERROR` and `TIMEOUT` are not set by `ComparisonService`. When those situations happen, the Execution Engine creates a `StudentReport` with the correct status and calls `addReport()` directly without using `compare()`.

6.2 Classes

`NormalizationMode` is an enum with three values: `STRICT`, `TRIM_WHITESPACE`, and `CASE_INSENSITIVE`. It is set once for each project and stored in the `PROJECT` table.

`ComparisonStatus` is an enum with five values: `PASS`, `FAIL`, `ERROR`, `COMPILE_ERROR`, and `TIMEOUT`. Every evaluation gets exactly one of these. `ERROR` is used when something unexpected happens, for example if the expected output file is not found.

`ComparisonResult` is the object returned after a comparison. It holds the status, the actual output from the student's program, the expected output, and a list of differing lines which is only filled when the status is `FAIL`. It also has `isPassed()` and `getSummary()` methods.

`ComparisonService` does the comparison. Its main method is `compare(actualOutput: String, expectedOutputPath: String, normMode: NormalizationMode): ComparisonResult` which reads the file, normalizes both strings, compares them, and returns a `ComparisonResult`. It has two private methods: `normalize(text: String, mode: NormalizationMode): String` which applies the normalization rules, and `computeDiff(actual: List<String>, expected: List<String>): List<String>` which finds the lines that are different.

`StudentReport` holds all the information about one student's evaluation. It stores `studentId`, `zipFilePath`, the `ComparisonResult`, any compile error message, a timestamp, and the normalization mode that was used. It also has a `toCSVRow()` method that returns one line with the columns: `studentId`, `status`, `normalizationMode`, `timestamp`, `compileError`, `actualOutputPreview`, and `diffLineCount`.

`ReportingService` collects `StudentReport` objects during the run. After each student is processed, the Execution Engine calls `addReport()` to add the report to the list. When all students are done, `saveToProject()` writes everything to the `EVALUATION_RESULT` table. It also has `getSummaryStats()` which returns an `AssignmentReport` with the total counts, and `exportCSV(outputPath: String): void` which writes all reports to a CSV file.

`AssignmentReport` is returned by `getSummaryStats()`. It holds `totalStudents`, `passCount`, `failCount`, and `errorCount` (which includes `COMPILE_ERROR` and `TIMEOUT` together). It also has `getPassRate(): double` which returns the pass rate as a number between 0.0 and 1.0.

6.3 Class Diagram

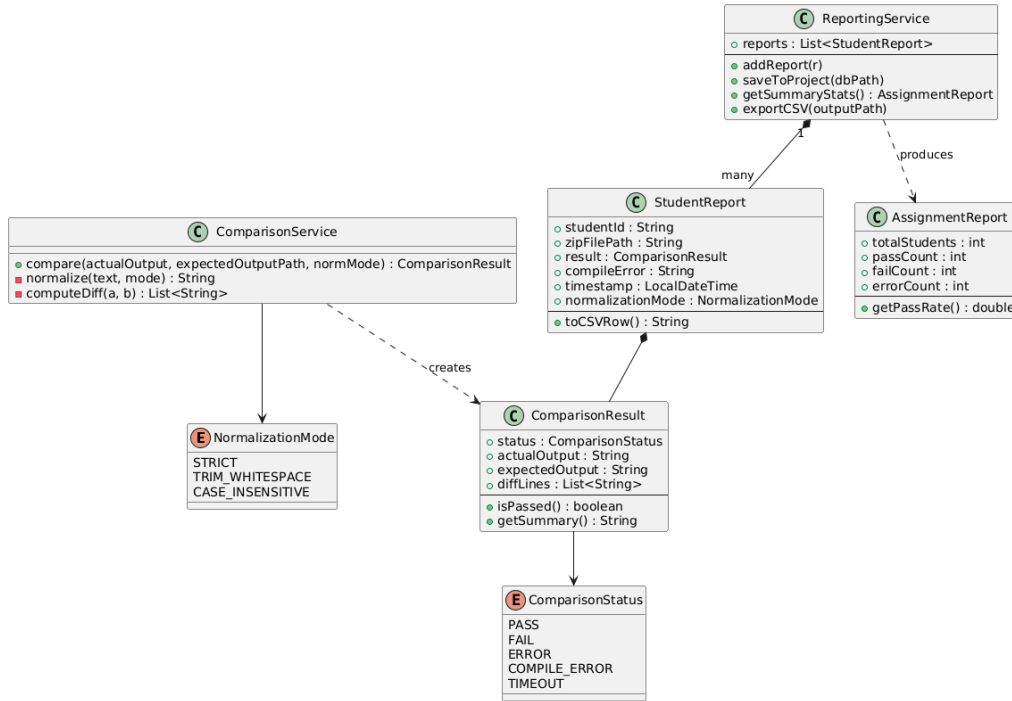


Figure 8: Evaluation and Reporting Class Diagram

7 UI and Deployment

This component covers the user interface, the Windows installer, and the Help menu. It covers Requirement 1 and Requirement 2. The application uses JavaFX. Because the target system has Java but not JavaFX, the JavaFX runtime is included in the installer.

The UI does not contain any business logic. It uses the other five components through their public methods. The **Controller** class connects UI actions to the right component calls.

Requirement 1 (system requirement): The software must be deployed to target Windows computers with an installer.

Requirement 2 (system requirement): The software must have a manual that will be displayed with a “Help” menu item.

7.1 Installer and System Deployment

We use `jlink` and `jpackage` from JDK 17 to make a package with the JVM, JavaFX, and our app’s JAR file. Then, we use Inno Setup to create a single `.exe` installer file.

The installer puts the app files in the normal Windows folders. The main app files, FXML, CSS, and HTML Help files go to `C:\Program Files\IAE`. We put some files in a specific `share/iae/` folder so the antivirus program won't block them.

The installer makes a desktop shortcut for all users on the computer, which is good for school labs. It also adds the app to the Start menu. The app saves the teachers' .iae project files (SQLite databases) in their Documents folder.

When the user uninstalls the app, it only deletes the files from Program Files. It does not delete the teachers' project files and results, so their work is always safe.

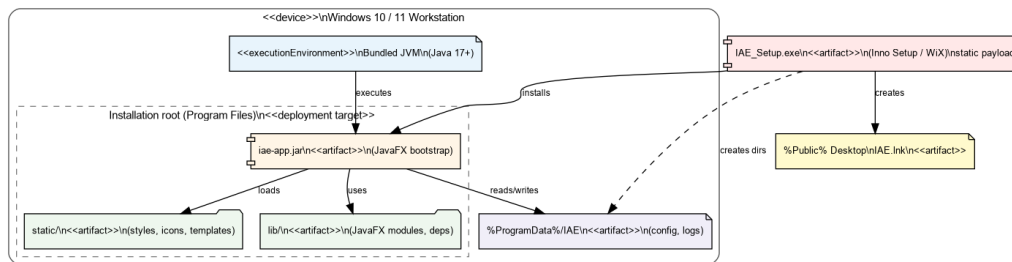


Figure 9: Deployment Diagram

7.2 Help

Our application includes a complete user manual that explains how to use the software step by step. Users can easily find and open this manual by clicking the “Help” menu item at the top of the screen.

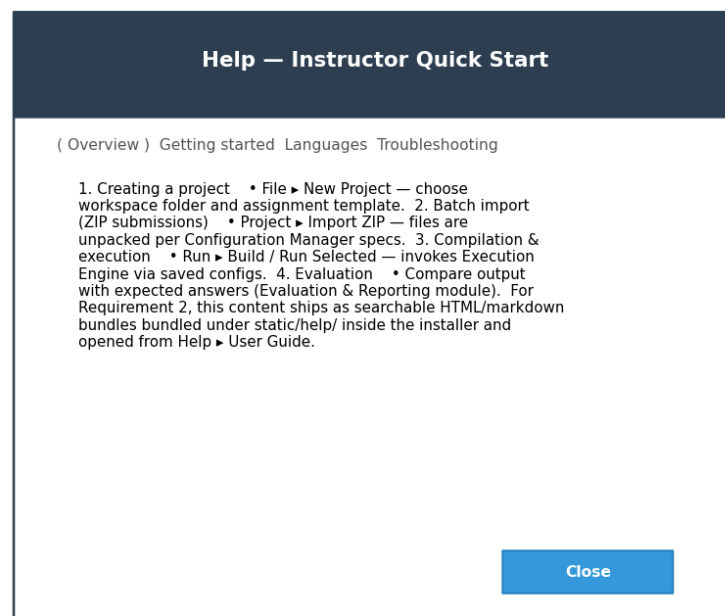


Figure 10: Help window referencing bundled documentation

7.3 Screens

The main window has a menu bar, a toolbar, and a content area in the middle. From there the lecturer can open the following screens.

The Project Creation Wizard collects the assignment name, the configuration, the folder with student ZIPs, the expected output file path, the run arguments (command-line parameters for the test cases), the timeout values for compilation and execution, and the normalization mode.

The Configuration Editor is used to create, edit, and delete `LanguageConfig` entries. It shows a validation message under each field as the lecturer types so problems are found right away.

The Import and Export Dialog shows a file picker and a summary after the operation is done. The summary lists how many configs were imported and which ones were skipped with a reason.

The Run Screen shows a progress bar for each student and a log that streams the stdout and stderr output as the Execution Engine works.

The Results Screen shows a summary bar at the top with the total, pass, fail, and error counts. Below is a table with one row per student. Each row shows the student ID, the status, the timestamp, and a short message. The lecturer can expand a row to see the full output and the diff. There is also an Export CSV button.

7.4 Controller

`Controller` holds references to all component services: `ConfigurationService`, `FileManager`, `Executioner`, `ComparisonService`, `ReportingService`, and `DatabaseService`. It has methods like `onNewProject()`, `onOpenProject()`, `onSaveProject()`, `onRunClicked()`, and `onExportCSV()`. When the lecturer clicks Run, `onRunClicked()` calls `FileManager.prepareSubmissions()` to get the list of submissions, passes it to `Executioner.executeAll()` together with the `ReportingService`, and then calls `ReportingService.saveToProject()` when the loop is done. The UI is updated with `Platform.runLater()` so the window stays responsive during the run.

7.5 UI Mockups

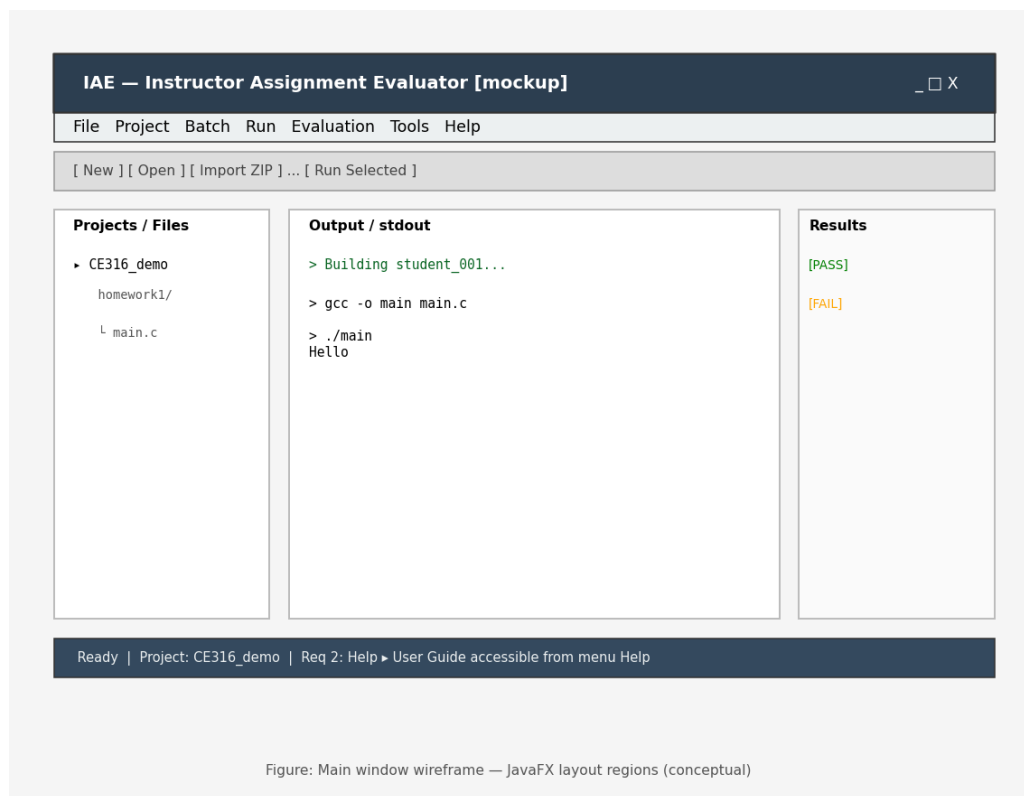


Figure 11: Main Application Shell

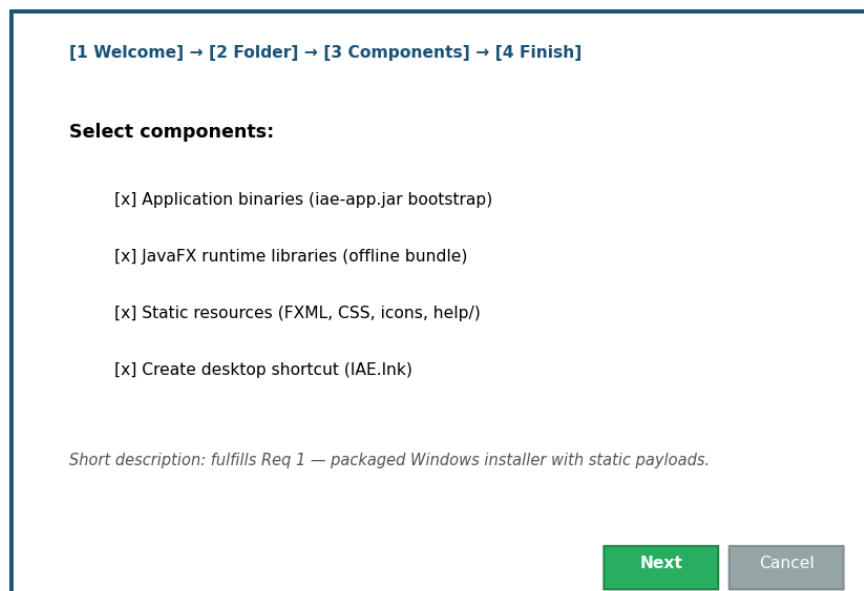


Figure 12: Installer component selection

7.6 Possible Problems and Risks

Our design meets all the requirements, but we might face some realistic problems when we start coding. Here are the risks and our solutions:

- **Windows Warnings:** Our installer (.exe) does not have a security certificate. Because of this, Windows Defender might show a warning message.
- **Folder Name Errors:** Some teachers might put their files in folders with special characters (like spaces or Turkish letters). To prevent file reading errors, we will use Java's safe file tools instead of just adding texts together.
- **Very Large Student Output:** If a student's code prints millions of lines by mistake, our app might run out of memory and freeze. To fix this, we will stop reading the output after 1 Megabyte (1 MB) and compare only that part.